

Master: Advanced Machine Learning and
Multimedia Intelligence

Evaluation of LLMs and VLMs
using Classical Approaches
and Reinforcement Learning

Project by:

Mohammed El Idrissi Laoukili

Mohamed Khalil Azroul

Introduction

The rapid advancement of artificial intelligence has led to the development of increasingly sophisticated language and vision models. This project explores comprehensive evaluation methodologies for two critical classes of AI systems: Large Language Models (LLMs) and Vision-Language Models (VLMs).

What are Large Language Models (LLMs)?

Large Language Models (LLMs) are deep neural networks trained on massive amounts of text data to understand and generate human language. These models, such as GPT-4, Claude, and LLaMA, contain billions of parameters and can perform a wide range of language tasks including:

- Text generation and completion
- Question answering
- Summarization
- Translation
- Code generation
- Reasoning and problem-solving

LLMs are typically based on the Transformer architecture and trained using self-supervised learning on diverse text corpora from the internet, books, and other sources. They learn statistical patterns in language that enable them to produce coherent, contextually appropriate responses.

Key characteristics of LLMs:

- Scale: Billions to trillions of parameters
- Pre-training: Trained on massive text datasets
- Few-shot learning: Can adapt to new tasks with minimal examples
- Emergent abilities: Complex behaviors arising from scale

What are Vision-Language Models (VLMs)?

Vision-Language Models (VLMs) are multimodal AI systems that combine visual understanding with language processing. These models, such as CLIP, DALL-E, GPT-4V, and LLaVA, can process both images and text to perform tasks that require understanding the relationship between visual and linguistic information.

Core capabilities of VLMs:

- Image captioning: Generating textual descriptions of images
- Visual question answering: Answering questions about image content
- Image-text retrieval: Finding images matching text queries or vice versa

- Visual reasoning: Understanding spatial relationships and abstract concepts
- Multimodal dialogue: Conversing about visual content

VLMs typically consist of three main components:

1. **Vision Encoder:** Processes images (e.g., ViT, ResNet)
2. **Language Model:** Processes and generates text (e.g., Transformer)
3. **Cross-modal Alignment:** Bridges vision and language representations

The Challenge of Evaluation

Evaluating LLMs and VLMs is fundamentally challenging because:

- **Subjective quality:** Human preferences for "good" outputs vary
- **Diverse tasks:** Models perform multiple tasks requiring different evaluation criteria
- **Open-ended generation:** Infinite possible valid outputs for a given input
- **Context dependency:** Performance varies with task complexity and domain

Traditional metrics like accuracy are insufficient for these complex models, necessitating sophisticated evaluation frameworks.

Project Scope and Objectives

This project investigates evaluation paradigms for both LLMs and VLMs across seven chapters:

For Large Language Models (Chapters 1-3):

1. Classical Approaches:

- Static metrics: BLEU, ROUGE, METEOR
- Embedding-based metrics: BERTScore, sentence similarity
- Reference-free metrics: Perplexity, MAUVE
- Task-specific metrics for classification, QA, and generation

2. Reinforcement Learning-Based Evaluation:

- Treating models as RL agents
- Reward-based assessment through human feedback (RLHF)
- Interactive and sequential task evaluation
- Long-horizon performance measurement

3. Hybrid Approaches:

- Combining classical metrics with RL frameworks
- Multi-objective evaluation strategies
- Cost-effective comprehensive assessment

For Vision-Language Models (Chapters 4-6):

4. Classical Multimodal Approaches:

- Image-text alignment metrics
- Visual grounding evaluation
- Cross-modal retrieval metrics
- Multimodal task-specific benchmarks

5. RL-Based Multimodal Evaluation:

- Visual reasoning with reward signals
- Interactive vision-language tasks
- Multimodal alignment with human preferences

6. Hybrid Multimodal Strategies:

- Combining visual and textual classical metrics with RL
- Cross-modal reward design
- Comprehensive VLM assessment

Unified Framework (Chapter 7):

- Integrated evaluation for both LLMs and VLMs
- Comparative analysis across model types
- Best practices and recommendations
- Future directions in AI evaluation

Motivation

As LLMs and VLMs become integral to real-world applications—from healthcare and education to creative tools and autonomous systems—robust evaluation becomes critical for:

- **Safety:** Ensuring models behave reliably and avoid harmful outputs
- **Alignment:** Verifying models follow human values and intentions
- **Performance:** Measuring task success across diverse scenarios
- **Comparison:** Enabling fair model benchmarking and selection
- **Development:** Guiding iterative improvement and debugging

Document Structure

This document is organized into seven comprehensive chapters, divided into three parts:

Part I: Evaluation of Large Language Models (LLMs)

- **Chapter 1:** Evaluation of LLMs using classical approaches—traditional metrics that assess surface-level quality, semantic similarity, and task-specific performance (BLEU, ROUGE, METEOR, BERTScore, Perplexity, MAUVE)
- **Chapter 2:** Evaluation of LLMs using reinforcement learning—treating language models as agents and evaluating based on cumulative rewards and alignment with human preferences through RLHF
- **Chapter 3:** Combining both approaches for LLMs—hybrid strategies that leverage the strengths of classical metrics and RL-based evaluation for comprehensive, cost-effective assessment

Part II: Evaluation of Vision-Language Models (VLMs)

- **Chapter 4:** Evaluation of VLMs using classical approaches—multimodal metrics for image-text alignment, visual question answering, and cross-modal retrieval tasks
- **Chapter 5:** Evaluation of VLMs using reinforcement learning—RL-based assessment of visual reasoning, interactive vision tasks, and multimodal alignment with human preferences
- **Chapter 6:** Combining both approaches for VLMs—hybrid evaluation strategies specifically designed for multimodal models that process both visual and linguistic information

Each chapter provides theoretical foundations, mathematical formulations, practical examples, and implementation considerations to create a complete understanding of modern LLM and VLM evaluation methodologies.

Chapter 1: Evaluation of Large Language Models (LLMs) Using Classical Approaches

1 What Are Classical Approaches?

Classical approaches are static, task-specific metrics that evaluate LLM outputs without dynamic interaction or learning. They contrast with learning-based methods like reinforcement learning.

2 Core Objectives

Classical evaluation assesses:

- Fluency: Is the output grammatically correct?
- Coherence: Does it make logical sense?
- Relevance: Does it address the prompt?
- Factuality: Is the information accurate?
- Diversity: Does it avoid repetition?

3 Evaluation Metrics

3.1 N-gram Overlap Metrics

3.1.1 BLEU (Bilingual Evaluation Understudy)

Equation:

$$\text{BLEU} = BP \cdot \exp \left(\sum_{n=1}^N w_n \log p_n \right) \quad (1)$$

where BP (Brevity Penalty) is:

$$BP = \begin{cases} 1 & \text{if } c > r \\ e^{1-r/c} & \text{if } c \leq r \end{cases} \quad (2)$$

c = candidate length, r = reference length, p_n = n-gram precision.

How it works: Measures how many word sequences (n-grams) from the candidate appear in the reference.

Example:

- Reference: "the cat sat on the mat"
- Candidate: "the cat is on the mat"
- Score: ≈ 0.6 (good match, but not perfect)

Best for: Machine translation

3.1.2 ROUGE (Recall-Oriented Understudy for Gisting Evaluation)

Equation (ROUGE-N):

$$\text{ROUGE-N} = \frac{\sum_{S \in \text{Ref}} \sum_{\text{n-gram} \in S} \text{Count}_{\text{match}}(\text{n-gram})}{\sum_{S \in \text{Ref}} \sum_{\text{n-gram} \in S} \text{Count}(\text{n-gram})} \quad (3)$$

How it works: Focuses on recall - whether important words from the reference appear in the candidate.

Example:

- Reference: "The dog ran fast"
- Candidate: "The fast dog ran"
- ROUGE-L finds: "The...dog ran" (captures key content despite different order)

Best for: Summarization

3.1.3 METEOR

Equation:

$$\text{METEOR} = (1 - \text{Penalty}) \cdot \frac{P \cdot R}{\alpha \cdot R + (1 - \alpha) \cdot P} \quad (4)$$

where:

$$\text{Penalty} = 0.5 \cdot \left(\frac{\text{chunks}}{\text{matched unigrams}} \right)^3 \quad (5)$$

P = precision, R = recall, $\alpha \approx 0.9$

How it works: Matches words using exact matches, stems, and synonyms. Penalizes fragmented matches.

Example:

- Reference: "The car is fast"
- Candidate: "The automobile is quick"
- METEOR recognizes: "car" $\approx$ "automobile", "fast" $\approx$ "quick"

Best for: Translation with paraphrasing

3.2 Embedding-Based Metrics

3.2.1 BERTScore

Equations:

$$\text{Precision} = \frac{1}{|x|} \sum_{i=1}^{|x|} \max_j \cos(x_i, y_j) \quad (6)$$

$$\text{Recall} = \frac{1}{|y|} \sum_{j=1}^{|y|} \max_i \cos(x_i, y_j) \quad (7)$$

$$\text{F1} = \frac{2 \cdot P \cdot R}{P + R} \quad (8)$$

where x_i, y_j are contextual embeddings.

How it works: Uses BERT to create contextual word embeddings, then computes similarity between candidate and reference embeddings.

Example:

- Reference: "The doctor examined the patient"
- Candidate: "The physician checked the sick person"
- BERTScore: High (understands semantic similarity)

Best for: Any text generation task

3.2.2 Sentence-BERT Cosine Similarity

Equation:

$$\text{Sim}(x, y) = \frac{\mathbf{e}_x \cdot \mathbf{e}_y}{\|\mathbf{e}_x\| \|\mathbf{e}_y\|} \quad (9)$$

How it works: Computes cosine similarity between sentence-level embeddings.

3.3 Reference-Free Metrics

3.3.1 Perplexity

Equation:

$$\text{PPL}(y) = \exp \left(-\frac{1}{T} \sum_{t=1}^T \log P(y_t | y_{<t}) \right) \quad (10)$$

How it works: Measures how surprised the model is by the text. Lower perplexity = more fluent text.

Example:

- Text 1: "The cat sat on the mat" → PPL: 20 (fluent)
- Text 2: "Cat the mat on sat the" → PPL: 500 (not fluent)

Important: Must use the same model that generated the text!

Best for: Measuring fluency

3.3.2 MAUVE Score

Equation:

$$\text{MAUVE} = (D_\lambda(P_{\text{human}} \| P_{\text{model}}) + D_\lambda(P_{\text{model}} \| P_{\text{human}}))^{-1} \quad (11)$$

How it works: Compares the distribution of human text vs. model-generated text using KL divergence.

Best for: Overall quality assessment of generative models

3.4 Task-Specific Metrics

- **Question Answering:** Exact Match (EM), F1 score

$$\text{F1} = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (12)$$

- **Text Classification:** Accuracy, Macro-F1
- **Code Generation:** Execution accuracy, pass@k

4 Limitations

- BLEU/ROUGE: Poor semantic understanding
- BERTScore: Still reference-dependent
- Perplexity: Not correlated with human judgment for open-ended generation
- All metrics: Fail to capture factuality, reasoning, and safety

5 When to Use Classical Metrics

- Benchmarking on standard datasets
- Quick iteration during development
- Baseline comparison with prior work
- Controlled settings with high-quality references

6 Summary Table

Metric	Best For	Score Range
BLEU	Translation	0–1
ROUGE	Summarization	0–1
METEOR	Translation/Paraphrase	0–1
BERTScore	Any generation	0–1
Perplexity	Fluency	1– ∞ (lower better)
MAUVE	Overall quality	0–1

Chapter 2: Evaluation of LLMs Using Reinforcement Learning

1 Why Use Reinforcement Learning for Evaluation?

Traditional metrics (perplexity, BLEU, ROUGE) fail to capture:

- Task success in interactive environments
- Alignment with human values
- Long-horizon coherence

RL provides a framework to:

- Define reward functions for desired behaviors
- Evaluate LLMs based on cumulative reward
- Simulate interactive scenarios

Key idea: Treat the LLM as an **agent** interacting with an **environment**, judged by **expected return**.

2 Core RL Framework

2.1 Basic Setup

Model the LLM as a stochastic policy $\pi_{\theta}(a|s)$ where:

- s = state (conversation history, prompt, environment observation)
- a = action (next token or sequence of tokens)
- θ = model parameters

Environment provides:

- Reward function: $r(s, a)$ or $r(s_t, a_t, s_{t+1})$
- Transition dynamics: $s_{t+1} = [s_t; a_t]$ (append action to state)

2.2 Key Definitions

Trajectory:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T) \quad (1)$$

Return:

$$R(\tau) = \sum_{t=0}^T \gamma^t r_t \quad (2)$$

where $\gamma \in [0, 1]$ is the discount factor.

Expected Return (Evaluation Objective):

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (3)$$

Interpretation: Higher $J(\pi_\theta)$ = better performance under RL evaluation.

3 Reward Design

3.1 Human Feedback as Reward (RLHF)

Train a reward model $r_\phi(s, a)$ from human preference data.

Training: Given two responses a_1, a_2 to prompt s , humans label which is better.

Loss function (Bradley-Terry model):

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(s, a_w) - r_\phi(s, a_l)) \quad (4)$$

where a_w = preferred response, a_l = rejected response, σ = sigmoid function.

Evaluation: Use r_ϕ to score LLM outputs, compute average reward over test prompts.

3.2 Rule-Based Rewards

Task-specific rewards:

- **Coding:** compile success + test case pass/fail
- **Math:** correctness verified by symbolic solver
- **Dialogue:** engagement, coherence, safety heuristics

Example:

$$r(s, a) = \begin{cases} +1 & \text{if answer is correct} \\ -1 & \text{if harmful content detected} \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

3.3 Learned Value Functions

Use a value network $V_\psi(s)$ to estimate future return.

Advantage function:

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s) \quad (6)$$

4 Evaluation Protocol

4.1 Monte Carlo Estimation

Practical steps:

1. Fix the LLM (π_θ is frozen)
2. Sample N responses to a set of prompts (states s_0)
3. Compute reward for each response (possibly multi-turn)
4. Report mean $\pm$ std of total return

Estimate:

$$\hat{J}(\pi_\theta) = \frac{1}{N} \sum_{i=1}^N R(\tau^{(i)}) \quad (7)$$

4.2 Policy Gradient Connection

Even without updating θ , the policy gradient theorem shows performance sensitivity:

$$\nabla_\theta J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot A^{\pi_\theta}(s_t, a_t) \right] \quad (8)$$

For evaluation:

- Sample trajectories
- Average returns
- Use baselines (e.g., $V(s_t)$) for variance reduction

5 Challenges

Challenge	Explanation
Sparse rewards	Many tasks give reward only at the end (e.g., code execution), making evaluation noisy
Reward hacking	LLM may exploit flaws in reward function (e.g., verbose answers to trigger keyword-based rewards)
Non-Markovian feedback	Human preferences depend on full context, not just current state-action
Cost	Requires either human labels or complex automated reward systems

6 When to Use RL-Based Evaluation

Most appropriate when:

- Task is interactive or sequential
- Success requires long-term planning (e.g., agent navigation)
- You have access to a meaningful reward signal (human or automated)

Less suitable for:

- Static text generation (e.g., summarization)
- Tasks without clear reward proxy

7 Key Equations Summary

$$\text{Policy: } \pi_{\theta}(a|s) \text{ --- LLM's token probability distribution} \quad (9)$$

$$\text{Return: } R(\tau) = \sum_{t=0}^T \gamma^t r_t \quad (10)$$

$$\text{Expected Return: } J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau)] \quad (11)$$

$$\text{Reward Model Loss: } \mathcal{L}_{\text{RM}} = -\log \sigma(r_{\phi}(s, a_w) - r_{\phi}(s, a_l)) \quad (12)$$

$$\text{Monte Carlo Estimate: } \hat{J}(\pi_{\theta}) = \frac{1}{N} \sum_{i=1}^N R(\tau^{(i)}) \quad (13)$$

8 Comparison with Classical Metrics

Aspect	Classical Metrics	RL-Based
Type	Static, reference-based	Interactive, reward-based
Evaluation	Single output	Full trajectory
Captures	Surface similarity	Task success, alignment
Cost	Low	High (rewards needed)
Use case	Standard benchmarks	Interactive tasks

Chapter 3: Combining Classical Approaches & Reinforcement Learning for LLM Evaluation

1 Why Combine Both Approaches?

Classical metrics and RL-based evaluation each have strengths and weaknesses:

Classical metrics:

- Fast and cheap to compute
- Good for surface-level quality (fluency, n-gram overlap)
- Limited: cannot capture task success or alignment

RL-based evaluation:

- Captures task performance and long-term behavior
- Aligns with human preferences
- Expensive: requires reward models or human feedback

Solution: Combine both to get comprehensive, cost-effective evaluation.

2 Integration Strategies

2.1 Strategy 1: Classical Metrics as Reward Components

Use classical metrics to define or augment the reward function in RL.

Composite reward function:

$$r(s, a) = \alpha \cdot r_{\text{task}}(s, a) + \beta \cdot r_{\text{classical}}(a) \quad (1)$$

where:

- $r_{\text{task}}(s, a)$ = task-specific reward (e.g., correctness)
- $r_{\text{classical}}(a)$ = classical metric score (e.g., BLEU, fluency)
- α, β = weighting hyperparameters

Example for translation:

$$r(s, a) = 1.0 \cdot \mathbb{I}[\text{correct}] + 0.3 \cdot \text{BLEU}(a, a_{\text{ref}}) - 0.2 \cdot \text{PPL}(a) \quad (2)$$

Benefits:

- Balances task success with output quality
- Provides denser reward signal (less sparse)
- Reduces reward hacking by penalizing low-quality text

2.2 Strategy 2: Multi-Objective Evaluation

Evaluate LLM on both classical metrics and RL returns separately, then combine.

Evaluation protocol:

1. Compute classical scores: $S_{\text{classical}} = \{\text{BLEU}, \text{ROUGE}, \text{BERTScore}, \dots\}$
2. Compute RL expected return: $J(\pi_\theta) = \mathbb{E}[R(\tau)]$
3. Normalize and combine:

$$\text{Final Score} = w_1 \cdot \bar{S}_{\text{classical}} + w_2 \cdot \bar{J}(\pi_\theta) \quad (3)$$

where $\bar{S}_{\text{classical}}$ and $\bar{J}(\pi_\theta)$ are normalized to $[0, 1]$.

Example weights:

- For creative writing: $w_1 = 0.3, w_2 = 0.7$ (favor RL/human preference)
- For translation: $w_1 = 0.6, w_2 = 0.4$ (favor classical metrics)

2.3 Strategy 3: Classical Metrics for Auxiliary Tasks

Use classical metrics to evaluate intermediate steps in RL trajectories.

Multi-step reward:

$$r_t = \begin{cases} \text{ROUGE}(a_t, \text{ref}) & \text{if } t < T \text{ (intermediate steps)} \\ r_{\text{final}}(s_T, a_T) & \text{if } t = T \text{ (final step)} \end{cases} \quad (4)$$

Use case: Multi-turn dialogue where each turn is evaluated for coherence (classical) and final outcome is task success (RL).

2.4 Strategy 4: Reward Model with Classical Features

Train the reward model using both human preferences and classical metric features.

Enhanced reward model:

$$r_\phi(s, a) = \text{NN}_\phi(\text{embed}(s, a), \text{BLEU}(a), \text{PPL}(a), \text{BERTScore}(a)) \quad (5)$$

Training:

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(s, a_w) - r_\phi(s, a_l)) + \lambda \cdot \text{MSE}(\text{features}) \quad (6)$$

Benefits:

- Reward model learns to correlate classical metrics with human preferences
- More robust to distribution shift
- Requires fewer human labels

3 Practical Framework

3.1 Hybrid Evaluation Pipeline

Step-by-step process:

1. **Generate outputs:** Sample N responses from LLM for test prompts
2. **Compute classical metrics:**
 - BLEU, ROUGE (if references available)
 - Perplexity (self-evaluation)
 - BERTScore (semantic similarity)
3. **Simulate RL episodes:**
 - Run multi-turn interactions
 - Collect trajectories $\tau^{(i)}$
 - Compute returns $R(\tau^{(i)})$
4. **Aggregate scores:**

$$\text{Score}_{\text{hybrid}} = \alpha \cdot \text{Mean}(S_{\text{classical}}) + \beta \cdot \text{Mean}(R(\tau)) \quad (7)$$

3.2 Example: Coding Assistant Evaluation

Task: Evaluate LLM’s ability to generate correct, readable code.

Classical component:

- Code similarity to reference (BLEU on tokenized code)
- Code perplexity (fluency)

RL component:

- Compilation success: $r_{\text{compile}} = +1$ if compiles, -1 otherwise
- Test cases passed: $r_{\text{test}} = \frac{\text{passed}}{\text{total}}$

Combined reward:

$$r_{\text{total}} = 2.0 \cdot r_{\text{test}} + 1.0 \cdot r_{\text{compile}} + 0.5 \cdot \text{BLEU}_{\text{code}} \quad (8)$$

4 Advantages of Hybrid Approach

Combining classical and RL-based evaluation provides several key benefits:

4.1 Comprehensive Coverage

Classical metrics capture:

- Surface-level quality (grammar, fluency)
- Lexical similarity to references
- Semantic coherence

RL metrics capture:

- Task completion and correctness
- Multi-turn coherence
- Alignment with human preferences

Together, they provide a complete picture of model performance.

4.2 Cost-Effectiveness

Classical metrics are cheap to compute and can filter out poor candidates before expensive RL evaluation.

Example workflow:

1. Run BLEU/ROUGE on 1000 candidates (fast, cheap)
2. Select top 100 based on classical scores
3. Run RL evaluation only on these 100 (expensive but focused)
4. Save 90% of RL evaluation cost

4.3 Reduced Reward Hacking

Classical metrics act as constraints that prevent gaming the reward function.

Example: An LLM might generate very long, verbose answers to maximize keyword-based rewards. Classical perplexity penalizes this, keeping outputs natural.

4.4 Better Interpretability

When a model fails, you can diagnose which component is problematic:

- Low BLEU, high RL return → Model solves task but uses different wording
- High BLEU, low RL return → Model produces fluent text but wrong answer
- Both low → Model fundamentally fails at the task

4.5 Flexibility Across Tasks

Adjust weights α, β based on what matters for each task:

- Creative writing: $\alpha = 0.2, \beta = 0.8$ (favor human preference)
- Translation: $\alpha = 0.7, \beta = 0.3$ (favor reference similarity)
- Code generation: $\alpha = 0.3, \beta = 0.7$ (favor correctness)

5 Challenges

5.1 Weight Selection Problem

Choosing optimal α, β weights is non-trivial.

Challenge: Different tasks require different balances.

Solutions:

- Use validation set to grid search over weight combinations
- Learn weights automatically via meta-learning
- Use multi-objective optimization (Pareto frontier)

Example: For machine translation, test $\alpha \in \{0.3, 0.5, 0.7\}$ and $\beta \in \{0.3, 0.5, 0.7\}$, pick combination that correlates best with human judgments.

5.2 Score Normalization

Classical metrics and RL returns live in different scales.

Problem:

- BLEU: $[0, 1]$
- RL return: Could be $[-100, +100]$ or any range

Solutions:

$$\bar{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad \text{or} \quad \bar{x} = \frac{x - \mu}{\sigma} \quad (9)$$

Use min-max normalization or z-score normalization before combining.

5.3 Conflicting Signals

Classical and RL metrics may disagree, making interpretation difficult.

Example scenario:

- Model generates: "The answer is 42 because of quantum fluctuations in spacetime."
- Classical BLEU: High (matches reference words)
- RL correctness: Low (wrong reasoning)

Solution: Analyze disagreement cases manually to understand failure modes. Use disagreement as a signal for model debugging.

5.4 Computational Overhead

Hybrid evaluation requires computing both types of metrics.

Cost breakdown:

- Classical metrics: $O(N)$ where N = number of samples
- RL evaluation: $O(N \times T)$ where T = trajectory length

Mitigation:

- Cache classical metric computations
- Use classical metrics as cheap pre-filter
- Parallelize RL simulations across GPUs

5.5 Hyperparameter Tuning Complexity

More hyperparameters to tune: α, β , normalization parameters, reward function components.

Challenge: Risk of overfitting to validation set.

Best practice:

- Use separate validation and test sets
- Keep hyperparameters simple (e.g., equal weights $\alpha = \beta = 0.5$ as baseline)
- Report sensitivity analysis: how much does score change with different weights?

6 When to Use Hybrid Evaluation

Best suited for:

- Tasks requiring both quality and correctness (e.g., code generation, math)
- Limited budget for human evaluation
- Need for interpretable evaluation (understand where model fails)
- Multi-objective optimization during training

Less suitable for:

- Pure creative tasks (poetry, fiction) where classical metrics are meaningless
- Tasks with perfect automated evaluation (e.g., chess moves)

7 Key Equations Summary

$$\text{Composite Reward: } r(s, a) = \alpha \cdot r_{\text{task}}(s, a) + \beta \cdot r_{\text{classical}}(a) \quad (10)$$

$$\text{Multi-Objective Score: } \text{Score} = w_1 \cdot \bar{S}_{\text{classical}} + w_2 \cdot \bar{J}(\pi_\theta) \quad (11)$$

$$\text{Enhanced Reward Model: } r_\phi(s, a) = \text{NN}_\phi(\text{embed}(s, a), \text{features}) \quad (12)$$

$$\text{Hybrid Evaluation: } \text{Score}_{\text{hybrid}} = \alpha \cdot \text{Mean}(S_{\text{classical}}) + \beta \cdot \text{Mean}(R(\tau)) \quad (13)$$

8 Comparison of All Approaches

Aspect	Classical	RL-Based	Hybrid
Cost	Low	High	Medium
Task alignment	Poor	Excellent	Good
Fluency capture	Excellent	Poor	Excellent
Interpretability	High	Low	High
Reward hacking	N/A	High risk	Low risk
Use case	Benchmarking	Interactive tasks	General purpose

9 Example Implementation

Pseudo-code for hybrid evaluation:

```
function evaluate_hybrid(model, test_set):
    classical_scores = []
    rl_returns = []

    for prompt in test_set:
        # Generate response
        response = model.generate(prompt)

        # Classical evaluation
        bleu = compute_bleu(response, reference)
        bertscore = compute_bertscore(response, reference)
        classical_scores.append((bleu + bertscore) / 2)

        # RL evaluation (multi-turn if applicable)
        trajectory = simulate_interaction(model, prompt)
        total_return = sum(trajectory.rewards)
        rl_returns.append(total_return)

    # Combine scores
    classical_avg = mean(classical_scores)
    rl_avg = mean(rl_returns)

    # Normalize to [0, 1]
    classical_norm = classical_avg # Already in [0, 1]
    rl_norm = (rl_avg - rl_min) / (rl_max - rl_min)

    # Weighted combination
    final_score = 0.4 * classical_norm + 0.6 * rl_norm

    return final_score
```

10 Conclusion

Hybrid evaluation combines the best of both worlds:

- Classical metrics provide fast, interpretable quality measures
- RL evaluation captures task success and alignment
- Together, they offer comprehensive, cost-effective LLM evaluation

Recommendation: Start with classical metrics for initial screening, then use RL-based evaluation for top candidates. Combine both for final model selection.

Chapter 4:

Evaluation of Vision-Language Models (VLMs) Using Classical Approaches

1.1 What Are Vision-Language Models?

Vision-Language Models (VLMs) process and generate text conditioned on visual inputs (images or videos). Unlike LLMs that operate on text alone, VLMs must:

- Encode visual features (objects, scenes, relationships)
- Align visual and textual representations in a shared embedding space
- Ground language in visual content

Classical evaluation for VLMs extends text-only metrics with **multimodal alignment** requirements.

1.2 Core Evaluation Objectives

Classical VLM evaluation assesses:

- **Visual grounding:** Does the text accurately describe visible content?
- **Hallucination:** Does the model invent non-existent visual elements?
- **Semantic alignment:** Are visual and textual embeddings coherent?
- **Task fidelity:** Does output satisfy task constraints (captioning, VQA, retrieval)?

1.3 Task-Specific Metrics

1.3.1 Image Captioning Metrics

CIDEr (Consensus-based Image Description Evaluation)

Equation:

$$\text{CIDEr}(c) = \frac{1}{N} \sum_{n=1}^N \frac{g_n(c) \cdot \text{TF-IDF}(c^{(n)})}{\|\text{TF-IDF}(c^{(n)})\|} \quad (1.1)$$

where $g_n(c)$ is the cosine similarity between candidate and reference TF-IDF vectors for n -grams.

How it works: Weights n -grams by inverse document frequency across the dataset, emphasizing distinctive descriptive terms.

Example:

- Image: *A red bicycle leaning against a brick wall*

- Candidate: “A bicycle is parked near a wall”
- Reference: “A red bike rests against brick wall”
- CIDEr: Moderate (captures objects but misses color/texture details)

Best for: Caption diversity and descriptiveness

SPICE (Semantic Propositional Image Caption Evaluation)

Equation:

$$\text{SPICE}(c, r) = \frac{2 \cdot P \cdot R}{P + R} \quad (1.2)$$

where P and R are precision/recall of scene graph tuples (objects, relations, attributes).

How it works: Parses captions into scene graphs, compares semantic tuples rather than surface tokens.

Example:

- Candidate scene graph: (bicycle, color, red), (bicycle, near, wall)
- Reference scene graph: (bike, color, red), (bike, against, wall), (wall, material, brick)
- SPICE captures partial match on objects/relations

Best for: Semantic correctness beyond lexical overlap

CLIPScore

Equation:

$$\text{CLIPScore}(I, C) = \max(0, \cos(e_I^{\text{CLIP}}, e_C^{\text{CLIP}})) \quad (1.3)$$

where e_I, e_C are CLIP image and text embeddings.

How it works: Measures cosine similarity between visual and textual CLIP embeddings—reference-free.

Example:

- Image: *Golden retriever playing fetch*
- Caption A: “A dog chasing a ball” $\rightarrow \text{CLIPScore} \approx 0.82$
- Caption B: “A cat sleeping on sofa” $\rightarrow \text{CLIPScore} \approx 0.21$

Best for: Reference-free evaluation; correlates with human judgments ($r \approx 0.73$)

1.3.2 Visual Question Answering (VQA)

VQA Accuracy

Equation:

$$\text{Acc}(a) = \min \left(\frac{\#\text{humans answering } a}{3}, 1 \right) \quad (1.4)$$

How it works: Requires ≥ 3 human annotators to agree on answer; soft scoring tolerates synonyms.

Example:

- Question: “What color is the car?”
- Human answers: [“red”, “red”, “crimson”, “red”]
- Prediction “crimson” $\rightarrow$ Accuracy = 0.75

Best for: Robustness to answer phrasing variations

Balanced Pair Accuracy

Equation:

$$\text{Balanced Acc} = \frac{1}{2|D|} \sum_{(q,i,i') \in D} \mathbb{I}[a(q,i) = y] + \mathbb{I}[a(q,i') \neq y] \quad (1.5)$$

where (i, i') are complementary images (e.g., yes/no pairs).

How it works: Penalizes language priors by requiring correct answers on *both* images of a complementary pair.

Example:

- Pair 1: Image with *zebra* $\rightarrow$ Q: “Is there a zebra?” $\rightarrow$ Ans: “yes”
- Pair 2: Same scene *without* zebra $\rightarrow$ Q: “Is there a zebra?” $\rightarrow$ Ans: “no”
- Model answering “yes” to both fails balanced accuracy

Best for: Measuring visual grounding (vs. language bias)

1.3.3 Image-Text Retrieval

Recall@K

Equation:

$$\text{Recall@K} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[\text{rank}(i) \leq K] \quad (1.6)$$

How it works: Proportion of queries where correct item appears in top- K retrieved results.

Typical values: Recall@1, Recall@5, Recall@10 reported jointly.

Best for: Cross-modal search quality

Mean Rank (MR) & Median Rank (MedR)

Equation:

$$\text{MR} = \frac{1}{N} \sum_{i=1}^N \text{rank}(i), \quad \text{MedR} = \text{median}(\{\text{rank}(i)\}_{i=1}^N) \quad (1.7)$$

How it works: Average/median position of correct match in ranked list.

Best for: Sensitivity to outliers (MedR more robust)

1.3.4 Zero-Shot Classification

Top-1 / Top-5 Accuracy

Equation:

$$\text{Top-}k = \frac{1}{N} \sum_{i=1}^N \mathbb{I}[y_i \in \text{top-}k \text{ predictions}] \quad (1.8)$$

How it works: Measures if ground-truth label appears in top k predicted classes using text prompts.

Prompt engineering impact:

- Naive prompt: “a photo of a {class}” $\rightarrow$ 72.1% top-1
- Ensemble prompt: 80 templates averaged $\rightarrow$ 76.4% top-1

Best for: Measuring visual recognition without fine-tuning

1.4 Limitations of Classical VLM Metrics

- CIDEr/SPICE: Require expensive human annotations; ignore visual plausibility
- CLIPScore: Biased toward CLIP’s training distribution; fails on compositional reasoning
- VQA Accuracy: Vulnerable to language priors (e.g., “Is this a dog?” $\rightarrow$ “yes” 80% of time)
- All metrics: Cannot detect subtle hallucinations (e.g., wrong object count, implausible relations)

1.5 When to Use Classical VLM Metrics

- Benchmarking on standard datasets (COCO, Flickr30k, VQAv2)
- Rapid iteration during model development
- Comparing against published baselines
- Tasks with well-defined ground truth (captioning, retrieval)

1.6 Summary Table

Metric	Best For	Score Range
CIDEr	Caption diversity	0–10
SPICE	Semantic correctness	0–1
CLIPScore	Visual-text alignment	0–1
VQA Acc	Visual QA	0–1
Recall@K	Retrieval	0–1
Top-1 Acc	Zero-shot classification	0–1

Chapter 5:

Evaluation of Vision-Language Models Using Reinforcement Learning

1.1 Why Use RL for VLM Evaluation?

Classical metrics fail to capture:

- Interactive grounding: Can the VLM maintain visual consistency across dialogue turns?
- Embodied reasoning: Can it plan actions in visual environments (e.g., navigation)?
- Long-horizon alignment: Does generated text remain faithful to visual context over extended interactions?

RL evaluation treats the VLM as an agent receiving visual observations and producing actions (text or decisions), judged by cumulative reward reflecting task success.

1.2 Core RL Framework for VLMs

1.2.1 State Representation

The state combines visual and linguistic context:

$$s_t = \left[\underbrace{f_{\text{vis}}(I_t)}_{\text{visual features}}, \underbrace{h_{t-1}}_{\text{dialogue history}}, \underbrace{g_t}_{\text{task goal}} \right] \quad (1.1)$$

where f_{vis} is a vision encoder (ViT, CLIP), h_{t-1} is recurrent history encoding, and g_t is task instruction.

1.2.2 Action Space

- Text generation: $a_t = \text{token}$ (discrete) or $a_t = \text{sequence}$ (structured)
- Embodied actions: $a_t \in \{\text{forward, left, right, stop}\}$ for navigation
- Multimodal outputs: $a_t = (\text{text}, \text{bounding_box})$ for referring expression tasks

1.2.3 Trajectory and Return

Trajectory with visual observations:

$$\tau = (I_0, s_0, a_0, r_0, I_1, s_1, a_1, r_1, \dots, I_T, s_T, a_T, r_T) \quad (1.2)$$

Return:

$$R(\tau) = \sum_{t=0}^T \gamma^t r_t \quad (1.3)$$

Expected return (evaluation objective):

$$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [R(\tau)] \quad (1.4)$$

1.3 Reward Design for VLMs

1.3.1 Visual Grounding Rewards

CLIP-Based Reward

Equation:

$$r_t^{\text{CLIP}}(I, a) = \cos(e_I^{\text{CLIP}}, e_a^{\text{CLIP}}) \quad (1.5)$$

How it works: Rewards textual outputs aligned with visual content via CLIP similarity.

Example (Visual Dialogue):

- Image: Kitchen with red apple on counter
- Turn 3 Q: “What fruit do you see?”
- A: “A green banana” $\rightarrow r_t^{\text{CLIP}} = 0.18$ (low)
- A: “A red apple” $\rightarrow r_t^{\text{CLIP}} = 0.89$ (high)

Object Detection Reward

Equation:

$$r_t^{\text{det}} = \frac{1}{|O_{\text{ref}}|} \sum_{o \in O_{\text{ref}}} \mathbb{I}[\text{IoU}(b_o^{\text{pred}}, b_o^{\text{gt}}) > 0.5] \quad (1.6)$$

where O_{ref} are objects mentioned in text, b are bounding boxes.

How it works: Uses off-the-shelf detector (e.g., DETR) to verify mentioned objects exist in image.

Best for: Reducing hallucinations in captioning/referring expressions

1.3.2 Task-Success Rewards

Embodied Navigation (e.g., ALFRED, Habitat)

Equation:

$$r_t = \begin{cases} +1.0 & \text{if task completed successfully} \\ +0.1 & \text{if subgoal achieved} \\ -0.01 & \text{per step (time penalty)} \\ -1.0 & \text{if collision/invalid action} \end{cases} \quad (1.7)$$

How it works: Sparse terminal reward + shaping rewards for intermediate progress.

Example trajectory:

- s_0 : Image of living room, goal “Put mug in sink”
- a_0 : “go to kitchen” $\rightarrow r_0 = -0.01$
- a_5 : “pick up mug” $\rightarrow r_5 = +0.1$ (subgoal)
- a_{12} : “put mug in sink” $\rightarrow r_{12} = +1.0$ (task complete)

Visual Dialogue Reward

Equation:

$$r_t^{\text{dialog}} = \alpha \cdot r_t^{\text{helpful}} + \beta \cdot r_t^{\text{grounded}} + \gamma \cdot r_t^{\text{engaging}} \quad (1.8)$$

where components derived from rule-based heuristics or learned reward models.

1.4 Evaluation Protocol

1.4.1 Monte Carlo Estimation with Visual Context

Practical steps:

1. Fix VLM policy π_θ (frozen weights)
2. Sample N episodes on held-out visual environments/tasks
3. For each episode:
 - Observe initial image I_0
 - Generate actions conditioned on visual+text history
 - Receive rewards from environment (simulated or human-rated)
4. Report mean $\pm$ std of total return:

$$\hat{J}(\pi_\theta) = \frac{1}{N} \sum_{i=1}^N R(\tau^{(i)}) \quad (1.9)$$

1.4.2 Human-in-the-Loop Reward Modeling

Train multimodal reward model $r_\phi(I, s, a)$ from human preferences:

Preference dataset collection:

- Show annotator: image I + dialogue history h + two candidate responses (a_w, a_l)
- Annotator selects preferred response a_w

Reward model loss (Bradley-Terry):

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(I, h, a_w) - r_\phi(I, h, a_l)) \quad (1.10)$$

Evaluation: Average predicted reward over test episodes:

$$\hat{J}_{\text{RM}}(\pi_\theta) = \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^T r_\phi(I_t^{(i)}, h_t^{(i)}, a_t^{(i)}) \quad (1.11)$$

1.5 Challenges Specific to VLM RL Evaluation

- Visual reward sparsity: Correct answer may be one of many valid descriptions; requires dense intermediate rewards
- Grounding ambiguity: Multiple valid interpretations of visual scene; reward must handle subjectivity
- Distribution shift: Test images differ from reward model training distribution; causes reward misalignment
- Cost: Human evaluation of visual grounding is expensive; requires hybrid automated/human approach

1.6 When to Use RL-Based VLM Evaluation

Most appropriate when:

- Task is interactive (visual dialogue, embodied navigation)
- Success requires temporal consistency across visual observations
- Ground truth is procedural (sequence of actions) rather than static output

Less suitable for:

- Single-image captioning with reference captions available
- Retrieval tasks with clear ranking metrics

1.7 Key Equations Summary

Policy:	$\pi_\theta(a s)$ — VLM’s action distribution conditioned on visual+text state
Return:	$R(\tau) = \sum_{t=0}^T \gamma^t r_t$
Expected Return:	$J(\pi_\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[R(\tau)]$
CLIP Reward:	$r_t^{\text{CLIP}} = \cos(e_I^{\text{CLIP}}, e_a^{\text{CLIP}})$
Reward Model Loss:	$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(I, h, a_w) - r_\phi(I, h, a_l))$
Monte Carlo Estimate:	$\hat{J}(\pi_\theta) = \frac{1}{N} \sum_{i=1}^N R(\tau^{(i)})$

1.8 Comparison: Classical vs. RL Evaluation for VLMs

Aspect	Classical Metrics	RL-Based Evaluation
Type	Static, reference-based	Interactive, reward-based
Input	Single image + prompt	Sequence of visual observations
Output judged	Single text	Full trajectory of actions
Grounding check	Post-hoc (CLIPScore)	Online via reward signal
Temporal coherence	Not measured	Explicitly rewarded
Cost	Low (automated)	High (reward engineering/human labels)
Best use case	Captioning, VQA benchmarks	Embodied AI, visual dialogue agents

Chapter 6:

Combining Classical Approaches & Reinforcement Learning for VLM Evaluation

1.1 Why Combine Both Approaches?

Classical metrics and RL-based evaluation each have strengths and weaknesses:

Classical metrics:

- Fast and cheap to compute
- Good for visual-text alignment (CLIPScore, SPICE)
- Limited: cannot capture interactive grounding or temporal consistency

RL-based evaluation:

- Captures task performance in embodied/interactive settings
- Aligns with human preferences for visual dialogue
- Expensive: requires reward models or human feedback

Solution: Combine both to get comprehensive, cost-effective VLM evaluation.

1.2 Integration Strategies

1.2.1 Strategy 1: Classical Metrics as Reward Components

Use classical metrics to define or augment the reward function in RL.

Composite reward function:

$$r(I, s, a) = \alpha \cdot r_{\text{task}}(s, a) + \beta \cdot r_{\text{visual}}(I, a) \quad (1.1)$$

where:

- $r_{\text{task}}(s, a)$ = task-specific reward (e.g., navigation success)
- $r_{\text{visual}}(I, a)$ = classical visual grounding metric (e.g., CLIPScore)
- α, β = weighting hyperparameters

Example for visual dialogue:

$$r(I, s, a) = 1.0 \cdot \mathbb{I}[\text{helpful}] + 0.5 \cdot \text{CLIPScore}(I, a) + 0.3 \cdot \text{SPICE}(a, a_{\text{ref}}) \quad (1.2)$$

Benefits:

- Balances task success with visual grounding quality
- Provides denser reward signal (less sparse)
- Reduces hallucination by penalizing visually inconsistent text

1.2.2 Strategy 2: Multi-Objective Evaluation

Evaluate VLM on both classical metrics and RL returns separately, then combine.

Evaluation protocol:

1. Compute classical scores: $S_{\text{classical}} = \{\text{CLIPScore}, \text{SPICE}, \text{CIDEr}, \dots\}$
2. Compute RL expected return: $J(\pi_\theta) = \mathbb{E}[R(\tau)]$
3. Normalize and combine:

$$\text{Final Score} = w_1 \cdot \bar{S}_{\text{classical}} + w_2 \cdot \bar{J}(\pi_\theta) \quad (1.3)$$

where $\bar{S}_{\text{classical}}$ and $\bar{J}(\pi_\theta)$ are normalized to $[0, 1]$.

Example weights:

- For image captioning: $w_1 = 0.7, w_2 = 0.3$ (favor classical metrics)
- For embodied navigation: $w_1 = 0.2, w_2 = 0.8$ (favor RL/task success)

1.2.3 Strategy 3: Classical Metrics for Auxiliary Tasks

Use classical metrics to evaluate intermediate steps in RL trajectories.

Multi-step reward:

$$r_t = \begin{cases} \text{CLIPScore}(I_t, a_t) & \text{if } t < T \text{ (intermediate steps)} \\ r_{\text{final}}(s_T, a_T) & \text{if } t = T \text{ (final step)} \end{cases} \quad (1.4)$$

Use case: Multi-turn visual dialogue where each turn is evaluated for visual grounding (classical) and final outcome is user satisfaction (RL).

1.2.4 Strategy 4: Reward Model with Classical Features

Train the reward model using both human preferences and classical metric features.

Enhanced reward model:

$$r_\phi(I, s, a) = \text{NN}_\phi(\text{embed}(I, s, a), \text{CLIPScore}(I, a), \text{SPICE}(a), \text{CIDEr}(a)) \quad (1.5)$$

Training:

$$\mathcal{L}_{\text{RM}} = -\log \sigma(r_\phi(I, s, a_w) - r_\phi(I, s, a_l)) + \lambda \cdot \text{MSE}(\text{features}) \quad (1.6)$$

Benefits:

- Reward model learns to correlate classical metrics with human preferences
- More robust to distribution shift across visual domains
- Requires fewer human labels

1.3 Practical Framework

1.3.1 Hybrid Evaluation Pipeline

Step-by-step process:

1. Generate outputs: Sample N responses from VLM for test images/prompts
2. Compute classical metrics:
 - CLIPScore, SPICE (if references available)
 - CIDEr (caption diversity)
 - VQA Accuracy (if applicable)
3. Simulate RL episodes:
 - Run multi-turn interactions or embodied tasks
 - Collect trajectories $\tau^{(i)}$ with visual observations
 - Compute returns $R(\tau^{(i)})$
4. Aggregate scores:

$$\text{Score}_{\text{hybrid}} = \alpha \cdot \text{Mean}(S_{\text{classical}}) + \beta \cdot \text{Mean}(R(\tau)) \quad (1.7)$$

1.3.2 Example: Visual Dialogue Agent Evaluation

Task: Evaluate VLM’s ability to answer questions about images across multiple dialogue turns.

Classical component:

- Visual-text alignment (CLIPScore between image and response)
- Semantic correctness (SPICE comparing to reference answers)

RL component:

- Dialogue coherence: $r_{\text{coherent}} = +1$ if response consistent with history, -1 otherwise
- User satisfaction: $r_{\text{satisfy}} = \frac{\text{rating}}{5}$ (from human feedback)

Combined reward:

$$r_{\text{total}} = 2.0 \cdot r_{\text{satisfy}} + 1.0 \cdot r_{\text{coherent}} + 0.5 \cdot \text{CLIPScore}(I, a) \quad (1.8)$$

1.4 Advantages of Hybrid Approach

Combining classical and RL-based evaluation provides several key benefits:

1.4.1 Comprehensive Coverage

Classical metrics capture:

- Visual-text alignment (CLIPScore)
- Semantic correctness (SPICE, CIDEr)
- Object detection accuracy

RL metrics capture:

- Task completion in embodied settings
- Multi-turn consistency in dialogue
- Alignment with human preferences

Together, they provide a complete picture of VLM performance.

1.4.2 Cost-Effectiveness

Classical metrics are cheap to compute and can filter out poor candidates before expensive RL evaluation.

Example workflow:

1. Run CLIPScore/SPICE on 1000 candidates (fast, cheap)
2. Select top 100 based on classical scores
3. Run RL evaluation only on these 100 (expensive but focused)
4. Save 90% of RL evaluation cost

1.4.3 Reduced Hallucination

Classical metrics act as constraints that prevent visual hallucination.

Example: A VLM might generate descriptions of objects not present in the image to maximize dialogue engagement. CLIPScore penalizes this, keeping outputs grounded in visual content.

1.4.4 Better Interpretability

When a VLM fails, you can diagnose which component is problematic:

- Low CLIPScore, high RL return → VLM solves task but descriptions don't match image
- High CLIPScore, low RL return → VLM describes image well but fails at task
- Both low → VLM fundamentally fails at visual grounding and task

1.4.5 Flexibility Across Tasks

Adjust weights α, β based on what matters for each task:

- Image captioning: $\alpha = 0.8, \beta = 0.2$ (favor classical metrics)
- Visual navigation: $\alpha = 0.1, \beta = 0.9$ (favor task success)
- Visual dialogue: $\alpha = 0.4, \beta = 0.6$ (balance both)

1.5 Challenges

1.5.1 Weight Selection Problem

Choosing optimal α, β weights is non-trivial.

Challenge: Different visual tasks require different balances.

Solutions:

- Use validation set to grid search over weight combinations
- Learn weights automatically via meta-learning
- Use multi-objective optimization (Pareto frontier)

Example: For visual dialogue, test $\alpha \in \{0.3, 0.5, 0.7\}$ and $\beta \in \{0.3, 0.5, 0.7\}$, pick combination that correlates best with human judgments.

1.5.2 Score Normalization

Classical metrics and RL returns live in different scales.

Problem:

- CLIPScore: $[0, 1]$
- RL return: Could be $[-100, +100]$ or any range

Solutions:

$$\bar{x} = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad \text{or} \quad \bar{x} = \frac{x - \mu}{\sigma} \quad (1.9)$$

Use min-max normalization or z-score normalization before combining.

1.5.3 Conflicting Signals

Classical and RL metrics may disagree, making interpretation difficult.

Example scenario:

- VLM generates: “The blue car is parked next to the red house.”
- Classical CLIPScore: High (matches some visual elements)
- RL correctness: Low (car is actually green, house is yellow)

Solution: Analyze disagreement cases manually to understand failure modes. Use disagreement as a signal for model debugging.

1.5.4 Visual Domain Shift

VLM performance varies across different visual domains (indoor, outdoor, synthetic, etc.).

Cost breakdown:

- Classical metrics: $O(N)$ where N = number of image-text pairs
- RL evaluation: $O(N \times T)$ where T = trajectory length

Mitigation:

- Stratify evaluation across visual domains
- Use domain-specific classical metrics when available
- Test RL policies on diverse visual environments

1.5.5 Hyperparameter Tuning Complexity

More hyperparameters to tune: α, β , normalization parameters, reward function components.

Challenge: Risk of overfitting to validation set.

Best practice:

- Use separate validation and test sets
- Keep hyperparameters simple (e.g., equal weights $\alpha = \beta = 0.5$ as baseline)
- Report sensitivity analysis: how much does score change with different weights?

1.6 When to Use Hybrid Evaluation

Best suited for:

- Tasks requiring both visual grounding and task success (e.g., visual dialogue, embodied AI)
- Limited budget for human evaluation
- Need for interpretable evaluation (understand where VLM fails)
- Multi-objective optimization during training

Less suitable for:

- Simple image captioning with good reference captions available
- Tasks with perfect automated evaluation (e.g., exact object counting)

1.7 Key Equations Summary

Composite Reward:	$r(I, s, a) = \alpha \cdot r_{\text{task}}(s, a) + \beta \cdot r_{\text{visual}}(I, a)$
Multi-Objective Score:	$\text{Score} = w_1 \cdot \bar{S}_{\text{classical}} + w_2 \cdot \bar{J}(\pi_\theta)$
Enhanced Reward Model:	$r_\phi(I, s, a) = \text{NN}_\phi(\text{embed}(I, s, a), \text{features})$
Hybrid Evaluation:	$\text{Score}_{\text{hybrid}} = \alpha \cdot \text{Mean}(S_{\text{classical}}) + \beta \cdot \text{Mean}(R(\tau))$

1.8 Comparison of All Approaches

Aspect	Classical	RL-Based	Hybrid
Cost	Low	High	Medium
Visual grounding	Good	Poor	Excellent
Task success	Poor	Excellent	Good
Interpretability	High	Low	High
Hallucination detection	Medium	Low	High
Use case	Benchmarking	Interactive tasks	General purpose

1.9 Example Implementation

Pseudo-code for hybrid VLM evaluation:

```
function evaluate_hybrid_vlm(model, test_set):
    classical_scores = []
    rl_returns = []

    for (image, prompt) in test_set:
        # Generate response
        response = model.generate(image, prompt)

        # Classical evaluation
        clip_score = compute_clip_score(image, response)
        spice = compute_spice(response, reference)
        classical_scores.append((clip_score + spice) / 2)

        # RL evaluation (multi-turn dialogue)
        trajectory = simulate_visual_dialogue(model, image, prompt)
        total_return = sum(trajectory.rewards)
        rl_returns.append(total_return)

    # Combine scores
    classical_avg = mean(classical_scores)
    rl_avg = mean(rl_returns)

    # Normalize to [0, 1]
    classical_norm = classical_avg # Already in [0, 1]
    rl_norm = (rl_avg - rl_min) / (rl_max - rl_min)

    # Weighted combination
    final_score = 0.5 * classical_norm + 0.5 * rl_norm

    return final_score
```

1.10 Conclusion

Hybrid evaluation combines the best of both worlds:

- Classical metrics provide fast, interpretable visual grounding measures
- RL evaluation captures task success and human alignment
- Together, they offer comprehensive, cost-effective VLM evaluation

Recommendation: Start with classical metrics for initial screening, then use RL-based evaluation for top candidates. Combine both for final VLM selection in deployment scenarios.

References

Classical Evaluation Metrics for Language Models

1. Papineni, K., Roukos, S., Ward, T., & Zhu, W. J. (2002). *BLEU: a method for automatic evaluation of machine translation*. In Proceedings of the 40th annual meeting of the Association for Computational Linguistics (pp. 311-318).
<https://aclanthology.org/P02-1040.pdf>
2. Lin, C. Y. (2004). *ROUGE: A package for automatic evaluation of summaries*. In Text summarization branches out (pp. 74-81).
<https://aclanthology.org/W04-1013.pdf>
3. Banerjee, S., & Lavie, A. (2005). *METEOR: An automatic metric for MT evaluation with improved correlation with human judgments*. In Proceedings of the ACL workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization (pp. 65-72).
<https://aclanthology.org/W05-0909.pdf>
4. Zhang, T., Kishore, V., Wu, F., Weinberger, K. Q., & Artzi, Y. (2020). *BERTScore: Evaluating text generation with BERT*. In International Conference on Learning Representations (ICLR).
<https://arxiv.org/abs/1904.09675>
5. Pillutla, K., Swayamdipta, S., Zellers, R., Thickstun, J., Welleck, S., Choi, Y., & Harchaoui, Z. (2021). *MAUVE: Measuring the gap between neural text and human text using divergence frontiers*. In Advances in Neural Information Processing Systems (NeurIPS).
<https://arxiv.org/abs/2102.01568>
6. Reimers, N., & Gurevych, I. (2019). *Sentence-BERT: Sentence embeddings using Siamese BERT-networks*. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP).
<https://arxiv.org/abs/1908.10084>

Reinforcement Learning and RLHF

7. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
8. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., ... & Lowe, R. (2022). *Training language models to follow instructions with human feedback*. In Advances in Neural Information Processing Systems (NeurIPS).
<https://arxiv.org/abs/2203.02155>

9. Christiano, P. F., Leike, J., Brown, T., Martic, M., Legg, S., & Amodei, D. (2017). *Deep reinforcement learning from human preferences*. In Advances in Neural Information Processing Systems (NIPS).
<https://arxiv.org/abs/1706.03741>
10. Stiennon, N., Ouyang, L., Wu, J., Ziegler, D., Lowe, R., Voss, C., ... & Christiano, P. F. (2020). *Learning to summarize with human feedback*. In Advances in Neural Information Processing Systems (NeurIPS).
<https://arxiv.org/abs/2009.01325>
11. Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal policy optimization algorithms*. arXiv preprint arXiv:1707.06347.
<https://arxiv.org/abs/1707.06347>

Large Language Models

12. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., ... & Polosukhin, I. (2017). *Attention is all you need*. In Advances in Neural Information Processing Systems (NIPS).
<https://arxiv.org/abs/1706.03762>
13. Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J. D., Dhariwal, P., ... & Amodei, D. (2020). *Language models are few-shot learners*. In Advances in Neural Information Processing Systems (NeurIPS).
<https://arxiv.org/abs/2005.14165>
14. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M. A., Lacroix, T., ... & Lample, G. (2023). *LLaMA: Open and efficient foundation language models*. arXiv preprint arXiv:2302.13971.
<https://arxiv.org/abs/2302.13971>
15. Anthropic. (2024). *Claude 3 model card*. Retrieved from Anthropic documentation.
16. OpenAI. (2023). *GPT-4 technical report*. arXiv preprint arXiv:2303.08774.
<https://arxiv.org/abs/2303.08774>

Vision-Language Models

17. Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., ... & Sutskever, I. (2021). *Learning transferable visual models from natural language supervision*. In International Conference on Machine Learning (ICML).
<https://arxiv.org/abs/2103.00020>
18. Ramesh, A., Dhariwal, P., Nichol, A., Chu, C., & Chen, M. (2022). *Hierarchical text-conditional image generation with CLIP latents*. arXiv preprint arXiv:2204.06125.
<https://arxiv.org/abs/2204.06125>
19. Liu, H., Li, C., Wu, Q., & Lee, Y. J. (2023). *Visual instruction tuning*. arXiv preprint arXiv:2304.08485.
<https://arxiv.org/abs/2304.08485>

20. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., ... & Houlsby, N. (2020). *An image is worth 16x16 words: Transformers for image recognition at scale*. In International Conference on Learning Representations (ICLR).
<https://arxiv.org/abs/2010.11929>

VLM Evaluation Metrics

21. Vedantam, R., Lawrence Zitnick, C., & Parikh, D. (2015). *CIDEr: Consensus-based image description evaluation*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
<https://arxiv.org/abs/1411.5726>
22. Anderson, P., Fernando, B., Johnson, M., & Gould, S. (2016). *SPICE: Semantic propositional image caption evaluation*. In European Conference on Computer Vision (ECCV).
<https://arxiv.org/abs/1607.08822>
23. Hessel, J., Holtzman, A., Forbes, M., Bras, R. L., & Choi, Y. (2021). *CLIPScore: A reference-free evaluation metric for image captioning*. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP).
<https://arxiv.org/abs/2104.08718>
24. Goyal, Y., Khot, T., Summers-Stay, D., Batra, D., & Parikh, D. (2017). *Making the V in VQA matter: Elevating the role of image understanding in visual question answering*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
<https://arxiv.org/abs/1612.00837>
25. Agrawal, A., Batra, D., Parikh, D., & Kembhavi, A. (2018). *Don't just assume; look and answer: Overcoming priors for visual question answering*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
<https://arxiv.org/abs/1712.00377>

Embodied AI and Interactive Evaluation

26. Shridhar, M., Thomason, J., Gordon, D., Bisk, Y., Han, W., Mottaghi, R., ... & Farhadi, A. (2020). *ALFRED: A benchmark for interpreting grounded instructions for everyday tasks*. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).
<https://arxiv.org/abs/1912.01734>
27. Savva, M., Kadian, A., Maksymets, O., Zhao, Y., Wijmans, E., Jain, B., ... & Batra, D. (2019). *Habitat: A platform for embodied AI research*. In Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV).
<https://arxiv.org/abs/1904.01201>
28. Das, A., Datta, S., Gkioxari, G., Lee, S., Parikh, D., & Batra, D. (2018). *Embodied question answering*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
<https://arxiv.org/abs/1711.11543>

Benchmarks and Evaluation Frameworks

- 29. Wang, A., Singh, A., Michael, J., Hill, F., Levy, O., & Bowman, S. R. (2018). *GLUE: A multi-task benchmark and analysis platform for natural language understanding*. In International Conference on Learning Representations (ICLR).
<https://arxiv.org/abs/1804.07461>
- 30. Wang, A., Pruksachatkun, Y., Nangia, N., Singh, A., Michael, J., Hill, F., ... & Bowman, S. R. (2019). *SuperGLUE: A stickier benchmark for general-purpose language understanding systems*. In Advances in Neural Information Processing Systems (NeurIPS).
<https://arxiv.org/abs/1905.00537>
- 31. Liang, P., Bommasani, R., Lee, T., Tsipras, D., Soylu, D., Yasunaga, M., ... & Koreeda, Y. (2022). *Holistic evaluation of language models*. arXiv preprint arXiv:2211.09110.
<https://arxiv.org/abs/2211.09110>
- 32. Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., ... & Zitnick, C. L. (2014). *Microsoft COCO: Common objects in context*. In European Conference on Computer Vision (ECCV).
<https://arxiv.org/abs/1405.0312>
- 33. Young, P., Lai, A., Hodosh, M., & Hockenmaier, J. (2014). *From image descriptions to visual denotations: New similarity metrics for semantic inference over event descriptions*. Transactions of the Association for Computational Linguistics, 2, 67-78.

Reward Modeling and Preference Learning

- 34. Bradley, R. A., & Terry, M. E. (1952). *Rank analysis of incomplete block designs: I. The method of paired comparisons*. Biometrika, 39(3/4), 324-345.
- 35. Bai, Y., Jones, A., Ndousse, K., Askell, A., Chen, A., DasSarma, N., ... & Kaplan, J. (2022). *Training a helpful and harmless assistant with reinforcement learning from human feedback*. arXiv preprint arXiv:2204.05862.
<https://arxiv.org/abs/2204.05862>
- 36. Ziegler, D. M., Stiennon, N., Wu, J., Brown, T. B., Radford, A., Amodei, D., ... & Irving, G. (2019). *Fine-tuning language models from human preferences*. arXiv preprint arXiv:1909.08593.
<https://arxiv.org/abs/1909.08593>

Multimodal Alignment and Cross-Modal Learning

- 37. Li, J., Li, D., Xiong, C., & Hoi, S. (2022). *BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation*. In International Conference on Machine Learning (ICML).
<https://arxiv.org/abs/2201.12086>

38. Jia, C., Yang, Y., Xia, Y., Chen, Y. T., Parekh, Z., Pham, H., ... & Sung, Y. H. (2021). *Scaling up visual and vision-language representation learning with noisy text supervision*. In International Conference on Machine Learning (ICML).
<https://arxiv.org/abs/2102.05918>
39. Chen, X., Fang, H., Lin, T. Y., Vedantam, R., Gupta, S., Dollár, P., & Zitnick, C. L. (2015). *Microsoft COCO captions: Data collection and evaluation server*. arXiv preprint arXiv:1504.00325.
<https://arxiv.org/abs/1504.00325>

Safety and Alignment

40. Ganguli, D., Lovitt, L., Kernion, J., Askill, A., Bai, Y., Kadavath, S., ... & Kaplan, J. (2022). *Red teaming language models to reduce harms: Methods, scaling behaviors, and lessons learned*. arXiv preprint arXiv:2209.07858.
<https://arxiv.org/abs/2209.07858>
41. Ji, Z., Lee, N., Frieske, R., Yu, T., Su, D., Xu, Y., ... & Fung, P. (2023). *Survey of hallucination in natural language generation*. ACM Computing Surveys, 55(12), 1-38.
<https://arxiv.org/abs/2202.03629>
42. Askill, A., Bai, Y., Chen, A., Drain, D., Ganguli, D., Henighan, T., ... & Kaplan, J. (2021). *A general language assistant as a laboratory for alignment*. arXiv preprint arXiv:2112.00861.
<https://arxiv.org/abs/2112.00861>